

TP CoAP pour l'IoT — Réponses

Environnement : Windows 11, Python 3.13.7, aiocoap 0.4.17. Tous les résultats ci-dessous ont été obtenus en lançant réellement `server.py`.

Fichiers

Fichier	Rôle
<code>server.py</code>	Serveur CoAP complet : <code>/temp</code> (observable), <code>/led</code> , <code>/logs</code> (avec DELETE), <code>/time</code> , <code>/sensors/room1/*</code> , <code>/biglog</code> , <code>/.well-known/core</code>
<code>observer.py</code>	Client d'observation scriptable, qui compte les notifications
<code>blockwise.py</code>	Télécharge <code>/biglog</code> en imposant la taille des blocs Block2
<code>client_gateway.py</code>	Client scriptable de la section 3.6 (logique d'une passerelle)

Écarts par rapport à l'énoncé (aiocoap 0.4.17 / Windows)

- `-e` et `-o` n'existent pas dans `aiocoap-client` 0.4.17. Il faut utiliser `--payload "..."` et `--observe`.
- `--block 32` n'existe pas. La seule option liée est `--payload-initial-szx`, qui ne concerne que Block1 (l'envoi). J'ai donc écrit `blockwise.py` pour imposer la taille des blocs Block2.
- Les codes de succès (2.xx) ne s'affichent pas par défaut. Seules les erreurs apparaissent. Pour voir 2.05 Content, 2.04 Changed, etc., il faut ajouter `-v`.
- `/.well-known/core` n'est pas généré automatiquement par Site : il faut ajouter `resource.WKResource(racine.get_resources_as_linkheader)`. L'attribut `ct=0` n'apparaît pas non plus, car les ressources ne déclarent pas de `ct`.
- Sous Windows, le serveur ne peut pas écouter sur `:::` (ValueError: The transport can not be bound to any-address). Le serveur écoute donc sur `localhost` sous Windows et sur `:::` ailleurs.
- Sous Windows, le serveur se bloque après la déconnexion d'un observateur (WinError 10054) : la boucle Proactor cesse alors de lire le socket UDP. Le serveur force donc la boucle `WindowsSelectorEventLoopPolicy`.
- La négociation des blocs n'est pas 64 → 128 → 256. Le serveur a répondu directement avec des blocs de 1024 octets (`szx=6`).

Module 1 : requêtes de base

Traces obtenues :

```

GET /time -> 2026-09-29T09:58:31Z (2.05 Content)
GET /temp -> 21.7 puis 24.8 (valeur qui change)
PUT /temp "19.0" -> 2.04 Changed ; GET juste après -> 19.0
GET /led -> off
PUT /led "on" -> 2.04 Changed ; GET -> on
PUT /led "peut-etre" -> 4.00 Bad Request "use 'on' ou 'off'"
POST /logs x2 -> 2.01 Created, Location: /logs/1 puis /logs/2
GET /inexistant -> 4.04 Not Found
DELETE /logs -> 2.02 Deleted ; GET -> (aucun log)

```

#	Question	Réponse
1	Code retour d'un GET sur /time ?	2.05 Content , avec l'heure UTC en payload (ex. 2026-09-29T09:58:31Z).
2	Code retour d'un PUT /temp à 19.0 ?	2.04 Changed . Un GET immédiat renvoie 19.0, mais la tâche de fond remplace la valeur au bout de 2 s au plus.
3	PUT /led avec « peut-etre » ?	4.00 Bad Request , payload use 'on' ou 'off'. render_put n'accepte que on ou off : toute autre valeur est une erreur du client.
4	Commande pour éteindre la LED ?	aiocoap-client -m put --payload "off" coap://localhost/led (-e "off" dans les versions plus récentes).
5	GET après 2 POST sur /logs ?	La liste numérotée et horodatée : 1: [11:58:36] ALERTE: Temperature elevee 35C 2: [11:58:36] ALERTE: Humidite faible 20% Chaque POST renvoie 2.01 Created avec Location-Path: /logs/1, puis /logs/2.
6	Code retour du DELETE sur /logs ?	2.02 Deleted . Un GET renvoie ensuite (aucun log).

Module 2 : l'observation

Trace de `python observer.py 10` :

```

réponse initiale : 2.05 Content 21.1 (Observe=0)
notification 1 : 25.0 (Observe=1)
notification 2 : 21.5 (Observe=2)
notification 3 : 23.3 (Observe=3)
notification 4 : 22.1 (Observe=4)
notification 5 : 24.1 (Observe=5)
=> 5 notifications en 10.0 s

```

Le numéro de l'option Observe augmente à chaque notification. C'est ce qui permet au client de repérer une notification perdue ou arrivée dans le désordre.

Calcul d'énergie (messages de 20 octets) : - Polling toutes les 1 s : 120 messages × 20 = **2 400 octets**. - Observation : 14 messages × 20 = **280 octets**. - L'observation émet donc environ **8,6 fois moins** de données (-88 %). Moins d'émissions radio, c'est une batterie qui dure plus longtemps.

#	Question	Réponse
1	Notifications en 10 s ?	5 (une toutes les 2 s), plus la réponse initiale.
2	Deux clients reçoivent-ils les notifications ?	Oui. Les deux observateurs lancés en même temps ont reçu exactement les mêmes valeurs, avec les mêmes numéros Observe (1 à 4 en 8 s). Le serveur garde une liste d'abonnés et notifie chacun.
3	Messages en polling vs observation sur 1 min ?	Polling : 120 messages (2 400 octets). Observation : 14 messages (280 octets).
4	Deux usages de l'observation dans une serre ?	1) Surveiller en continu la température et l'humidité pour déclencher la ventilation ou la brumisation. 2) Recevoir une alerte immédiate quand l'humidité du sol passe sous un seuil (arrosage), ou quand une porte s'ouvre ou le niveau d'une cuve change.
5	Méthode qui déclenche une notification ?	<code>self.updated_state()</code> , appelée dans <code>background_task()</code> et dans <code>render_put()</code> de <code>TemperatureResource</code> .

Module 3 : découverte et blockwise

`/ .well-known/core` obtenu :

```
</ .well-known/core>;ct="40",</temp>;obs,</led>,</logs>,</time>,</sensors/room1/temperature>,</sensors/room1/humidity>,</sensors/room1/light>,</biglog>,<https://christian.amsuess.com/tools/aiocoap/#version-0.4.17>;rel="impl-info"
```

Blockwise (`python blockwise.py <taille>`):

```
1024 octets : 2 blocs (1024 + 760)  <- choix par défaut du serveur (szx=6)
256 octets : 7 blocs (6 x 256 + 248)
32 octets : 56 blocs (55 x 32 + 24)
```

#	Question	Réponse
1	Nombre de ressources après ajout de <code>/sensors/room1/...</code> ?	8 ressources applicatives : <code>/temp</code> , <code>/led</code> , <code>/logs</code> , <code>/time</code> , les 3 <code>/sensors/room1/*</code> et <code>/biglog</code> . L'annuaire contient aussi <code>/ .well-known/core</code> lui-même, soit 9 entrées, plus un lien <code>impl-info</code> ajouté par aiocoap. Sans <code>/biglog</code> (juste après l'étape 3.2), il y en a 7.
2	Que signifie <code>;obs</code> ?	La ressource est observable : un client peut s'y abonner (GET avec <code>Observe=0</code>) pour recevoir des notifications. Seule <code>/temp</code> l'est.
3	Taille de <code>/biglog</code> et nombre de blocs en 256 ?	1 784 octets , soit 7 blocs de 256 octets (6 pleins + 1 de 248). Par défaut, le serveur a envoyé 2 blocs de 1024. En 32 octets, il en faut 56.

#	Question	Réponse
4	Pourquoi le blockwise est-il essentiel pour une caméra IoT ?	Une image (plusieurs dizaines de Ko) dépasse largement la MTU (~1280 octets en IPv6, et bien moins en 6LoWPAN/LoRa). Sans découpage, il faudrait fragmenter au niveau IP : un seul fragment perdu oblige à tout renvoyer. Avec le blockwise, CoAP découpe au niveau applicatif en blocs numérotés, chacun acquitté et retransmis seul. La mémoire tampon du capteur reste petite et on peut reprendre un transfert interrompu.
5	Avantage du multicast pour 200 capteurs ?	Une seule requête (par ex. <code>GET coap://[ff02::1]/.well-known/core</code>) interroge tous les capteurs en même temps. On les découvre et on les inventorie automatiquement, sans connaître ni configurer 200 adresses. On peut aussi envoyer une commande de groupe (tout éteindre, changer la configuration) en un seul message au lieu de 200.